

AADK 10: Task 2 — Functional Programming with Lambdas and Higher-Order Functions

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 10

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Concept Notes

Higher-Order Function

A **higher-order function** is a function that **takes another function as a parameter** or **returns a function** as a result. This allows developers to reuse logic and write more flexible and modular code.

Example:

```
fun calculate(a: Int, b: Int, operation: (Int, Int) -> Int): Int {  
    return operation(a, b)  
}
```

Lambda Expression

A **lambda expression** is a short anonymous function that can be passed as a value.

Example:

```
val add = { a: Int, b: Int -> a + b }
```

Lambdas are often used with collections and higher-order functions.

Function Type

A **function type** describes the type of a function including its parameters and return type.

Example:

```
(Int, Int) -> Int
```

Meaning:

- Takes **two integers**
- Returns **one integer**

Example variable with function type:

```
val multiply: (Int, Int) -> Int = { a, b -> a * b }
```

Inline Functions

An **inline function** tells the Kotlin compiler to insert the function's code directly where it is called. This reduces overhead when using lambdas and improves performance.

Example:

```
inline fun execute(action: () -> Unit) {  
    action()  
}
```

Lambda Example — Filter a List of Integers

```
val numbers = listOf(1, 5, 10, 15, 20)
```

```
val filteredNumbers = numbers.filter { it > 10 }
```

```
println(filteredNumbers)
```

Output:

```
[15, 20]
```

Higher-Order Function Example

```
fun operate(a: Int, b: Int, operation: (Int, Int) -> Int): Int {  
    return operation(a, b)  
}
```

```
val result = operate(5, 3) { x, y -> x + y }
```

```
println(result)
```

Output:

```
8
```

2. Design a Mini Use Case

Scenario

Student Grade Evaluation System

We want to evaluate students based on different grading rules using lambdas.

Data Model

```
data class Student(  
    val name: String,  
    val marks: Int  
)
```

Higher-Order Function

```
fun filterStudents(  
    students: List<Student>,  
    condition: (Student) -> Boolean  
): List<Student> {  
    return students.filter(condition)  
}
```

Pseudo Code

1. Create a list of students
2. Pass a lambda that checks if marks ≥ 50
3. Higher-order function filters students
4. Display passed students

Example Implementation:

```
val students = listOf(  
    Student("Rajat", 78),  
    Student("Aman", 45),  
    Student("Rahul", 65)  
)  
  
val passedStudents = filterStudents(students) { it.marks >= 50 }  
  
println(passedStudents)
```

Output:

[Rajat, Rahul]

3. Compose Lambda Usage Research

Lambdas are widely used in **Jetpack Compose** for handling events and UI behavior.

Common Places Where Lambdas Are Used

1 Button Click Handling

```
Button(onClick = { })
```

2 Modifier.clickable

```
Modifier.clickable { }
```

3 LazyColumn item rendering

```
items(list) { item -> }
```

Example Explanation

Example:

```
Button(onClick = { total++ }) {  
    Text("Add")  
}
```

Explanation:

- `onClick` expects a **lambda function**
- When the button is pressed:
 - The lambda executes
 - The total state variable increases
- Compose automatically **recomposes the UI**

Compose Example Snippet

```
var total by remember { mutableStateOf(0) }
```

```
Button(onClick = { total++ }) {  
    Text("Add")  
}
```

```
Text("Total: $total")
```

This updates the UI whenever the button is clicked.

4. Function Type Exercise

Define a Function Type Alias

```
typealias DiscountRule = (Double) -> Double
```

This represents a function that:

- Takes a **Double**
 - Returns a **Double**
-

Calculation Function

```
fun applyDiscount(price: Double, rule: DiscountRule): Double {  
    return rule(price)  
}
```

Using Different Lambdas

```
val studentDiscount: DiscountRule = { price -> price * 0.8 }  
val festivalDiscount: DiscountRule = { price -> price * 0.7 }
```

```
println(applyDiscount(1000.0, studentDiscount))  
println(applyDiscount(1000.0, festivalDiscount))
```

Output:

800

700

Advantage

Using function types allows developers to **change business logic dynamically** without modifying the core function. This improves flexibility, reusability, and maintainability compared to hard-coding logic inside the function.

5. Reflection

Lambdas and higher-order functions make Kotlin code more concise and reusable. They allow developers to pass behavior as parameters instead of repeating similar code. This improves readability and modularity. In Jetpack Compose, lambdas are essential for handling user interactions such as button clicks and gestures. They simplify event handling by allowing developers to write short, inline logic directly where it is needed in the UI.